Insurance Suite

Complete project explanation

A policy, billing and claims system for private car insurance in India, built step by step as a learning project from *Insurance-Suite-Implementation-Document.pdf*.

Language and runtime	TypeScript (strict, ESM) on Node.js 22
Database	PostgreSQL 17 (real server) or PGlite (PostgreSQL inside Node); plain SQL migrations
API	Fastify REST, versioned /v1, OpenAPI 3.1 generated from the route list
Screens	React + Vite (quote wizard, policy file)
Tests	Vitest: 497 tests in 47 files, all passing on PGlite and on real PostgreSQL
Size	4 business modules + platform, 23 migrations, 34 API routes, 8 configuration families
Status	All 29 roadmap steps done; extension Tier 1 items 1 and 2 done (items 3 and 4 next)

Contents

- 1. What we are building and why
- 2. The architecture diagram
- 3. The modules
- 4. The key ideas (effective dating, jobs, rating, underwriting, events)
- 5. The life of one policy
- 6. Design rules that are always kept
- 7. API, sign-in and screens
- 8. Configuration, database and tests
- 9. Running and deploying
- 10. What is done, what is missing

1. What we are building and why

An insurer sells a promise: *if your car is damaged, we pay*. To keep that promise it must remember who is covered, for what, and from which date; charge the right price; collect the money; and pay claims correctly. Insurance Suite does these four jobs in four modules that cooperate but never reach into each other's data.

One example. Asha Rao in Hyderabad wants to insure her 2023 Swift. An agent opens an **account** for her and starts a **submission** (a job). The **rating engine** prices third-party, own-damage and personal-accident covers from rate tables. **Underwriting rules** may refer an unusual risk to an underwriter. When she accepts, the job is **bound**: the quote becomes policy version 1 and is **issued** with a schedule PDF. Billing turns the premium into charges and invoices (annual or instalments). In month 4 she raises the insured value: a **policy change** creates version 2 with **slices** and a prorated extra premium. In month 7 she reports an accident: Claims asks Policy *what the policy looked like on the loss date*, and opens exposures, reserves and payments. At the end of the year the policy **renews** as a new term of the same policy, with the claim history in mind.

Why four modules? Each has a different owner, rate of change and set of rules in a real insurer. Keeping them apart (own tables, public doors, events) means one can change without breaking the others, and later could even become a separate service.

2. The architecture diagram

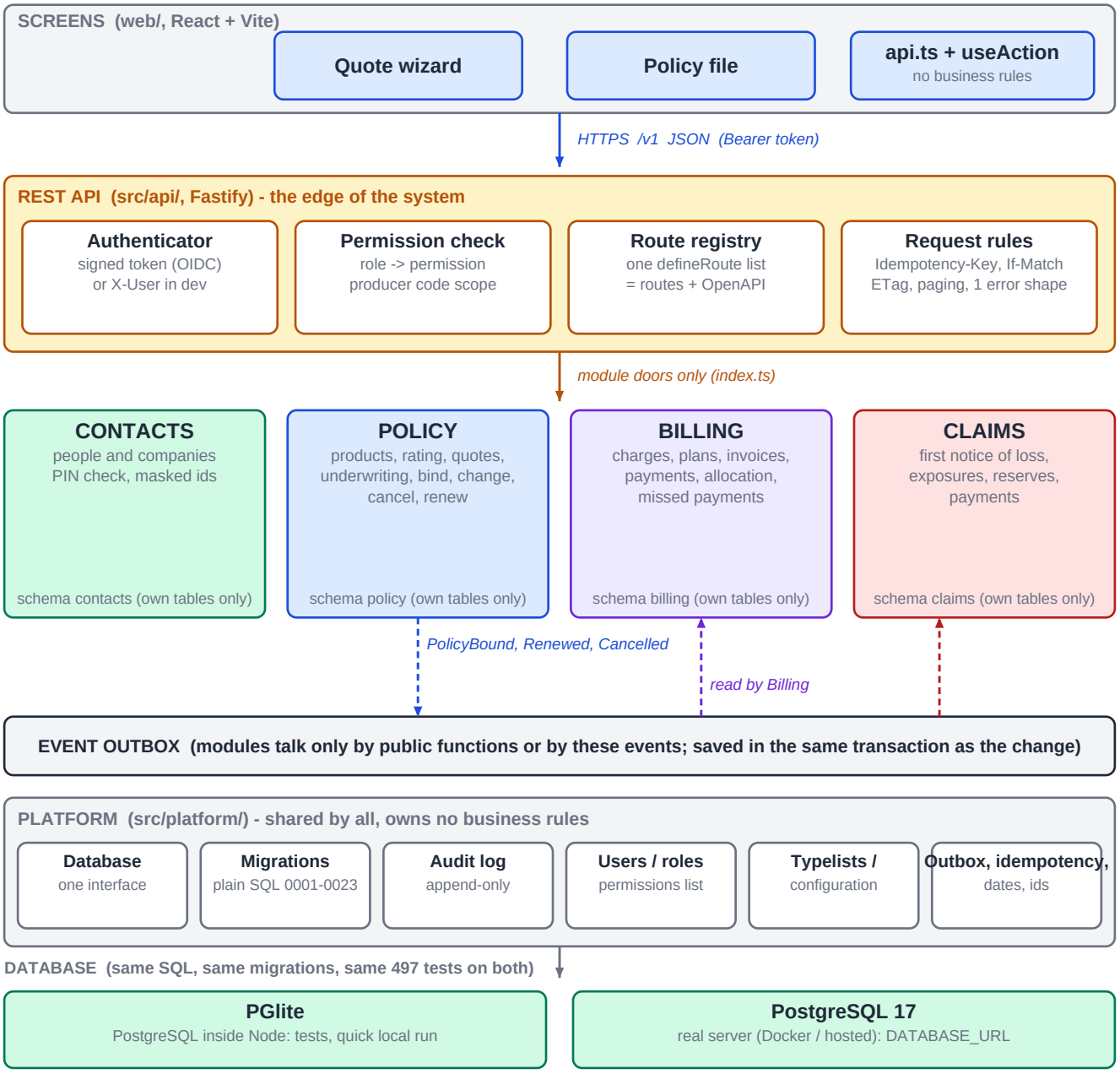


Figure 1. Layers: screens call the API; the API calls modules only through their public doors; modules share the platform and one database. Dashed arrows are events through the outbox, the only way Policy tells Billing something.

3. The modules

Module	Owns	Main ideas	Talks to
Contacts src/contacts	People and companies, addresses, identifiers	PIN code check; Aadhaar/PAN stored but always shown masked; never in logs or events	Used by Policy (named insured) and Claims (claimants)
Policy src/policy	Accounts, policies, terms, versions, jobs, quotes, underwriting issues, documents, cancellations, renewals	Effective dating, branches and slices, rating, underwriting, bind and issue, change, out-of-sequence merge, cancel, reinstate, renew	Publishes events; answers Claims with the policy as of a date; reads Contacts and Platform
Billing src/billing	Billing accounts, charges, payment items, invoices, payments, allocations, credits	Plans (annual, instalments), exact splits, oldest-due-first allocation, delinquency steps from configuration	Reads Policy events; sends CancellationRequested / ReinstatementRequested events back
Claims src/claims	Claims, exposures, reserve changes, payments, recoveries	First notice of loss, the policy snapshot kept with the claim, exposure life cycle, reserve and payment authority limits, second-person approval	Asks Policy via its public door; keeps what it received
Platform src/platform	Users, roles, producers, audit log, typelists, outbox, idempotency, migrations, configuration loading	No business rules; every change audited in the same transaction; permissions are a fixed list, roles are data	Used by everyone

Inside Policy (the biggest module)

Area	Files	What it does
Product model	product-model, product-catalogue, product-selection, expression	Versioned JSON products: coverages, terms, availability rules in a small safe expression language (never eval)
Effective dating	policy-repository, as-of, slice-rating	Policy, term, branch, slices; read 'as of a date'; rows never overlap (database exclusion constraint)
Quoting and rating	submission, quote, rating, rate-book, risk	Pure rating function from rate books; worksheet per cost; golden tests; several quote options
Underwriting	uw-rules, underwriting	Rules create issues; authority profiles decide who may approve; decisions are never edited
Bind and issue	bind, schedule, job-sync, job-status	Only bind makes a version 'bound'; premium transaction recorded; schedule PDF stored with SHA-256
Change	policy-change, merge	Copy the latest bound version, edit by slices, out-of-sequence protection and merge conflicts, preemption and rebase
Cancel, reinstate, renew	cancellation, cancellation-rules, refund, renewal, renewal-rules	Flat, pro rata and short-rate refunds; reinstatement window; renewal as a new term; non-renewal needs a second approver

4. The key ideas

Effective dating and versions

A policy is not one record that is overwritten. It is a **timeline**. Every change creates a new **version** (a **branch** of the period). Inside a version, each object (a coverage, the vehicle) is stored as **slices**, each valid from one date up to (not including) another. To know what applied on any day, read the slice whose range contains that day. A bound version is never edited; a change copies it into a working branch, edits that, prices it, and binds it as the next version.

One policy period (1 Jan - 1 Jan) and the versions that wrote its timeline

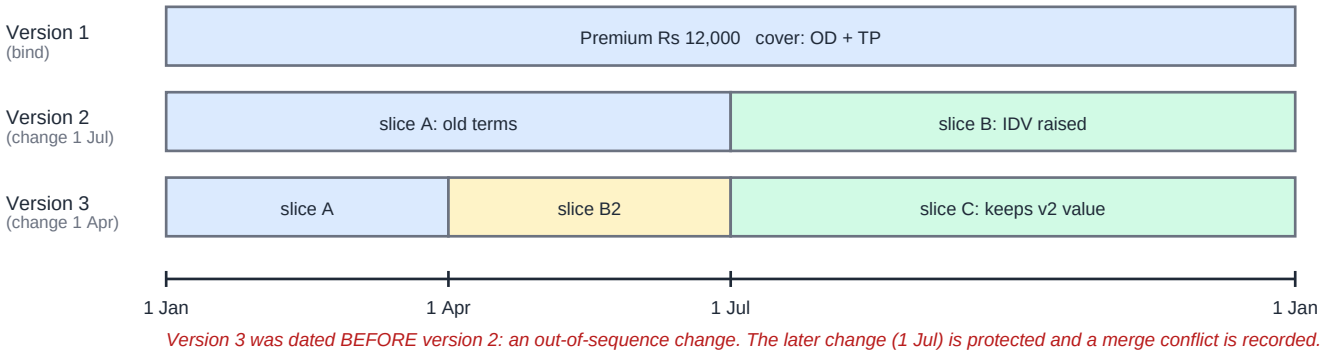


Figure 2. Versions and slices on one policy period.

Jobs

A **job** is a unit of work on a policy: submission, change, cancellation, reinstatement, renewal. A job has a status (Draft, Quoted, Bound, Issued, Withdrawn, Preempted...) and every allowed move is listed and tested. When one job binds, other open jobs on the same policy become **preempted** and must be rebased, never silently overwritten.

Rating

Premium comes from a pure function over a **rate book** (a versioned JSON file): base rates, factors (age of vehicle, engine size, claim-free years), discounts, taxes, in whole paise and basis points. Each cost is shown on a worksheet so a person can see why a price is what it is. A change is rated per slice and prorated by days. Golden tests pin known customers to exact prices.

Underwriting

Rules (also configuration) examine the facts of the risk and raise **issues**: for example, a very high insured value. An issue blocks quoting, binding or issuing at the point the rule names until someone with enough **authority** decides it. The decision is stored forever.

Events and the outbox

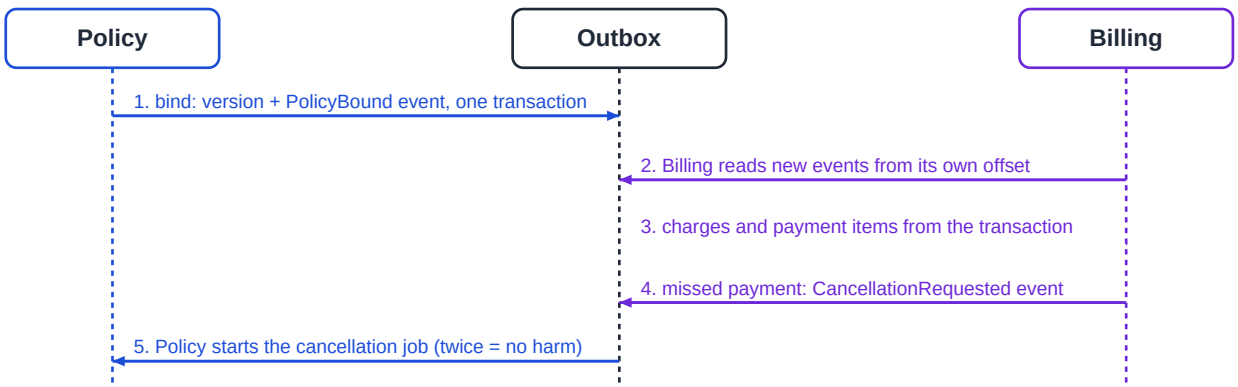
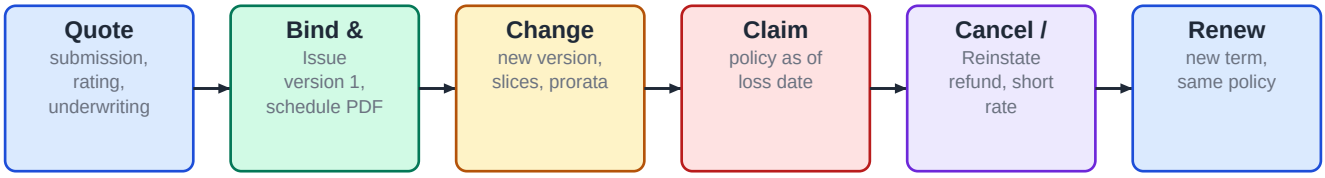


Figure 3. Policy and Billing cooperate through events. The event is saved in the same database transaction as the change that caused it, so a change can never happen without its event, or the reverse. Consumers must cope with seeing an event twice.

Audit and security

Every create or change writes an audit entry (who, when, what) in the same transaction; the audit table refuses updates, deletes and truncates. Permissions are a fixed list in code (bind, pay, handle claim...), roles are data an administrator can edit, and visibility of accounts is limited by **producer code** (an agent sees only their own business).

5. The life of one policy



Billing reacts to Bound, Changed, Renewed, Cancelled and Reinstated events: charges, invoices, payments.
A missed payment makes Billing send CancellationRequested; Policy starts the cancellation job.

Figure 4. The main flow, and which module acts at each step.

Step	API call (examples)	What happens inside	Result
Open account	POST /v1/contacts, POST /v1/accounts	Contact created; account tied to a producer code, checked through Platform	Account
Start quote	POST /v1/submissions, then vehicles and coverages	Policy, job and branch (Option A) are created; product version in force is recorded	Draft job
Quote	POST /v1/jobs/{id}/quote	Underwriting checked, rating run, quote row written (never updated)	Quoted job, price worksheet
Bind and issue	POST /v1/jobs/{id}/bind and /issue	Quote validity and underwriting checked; branch becomes version 1; PolicyBound event; schedule PDF stored	Policy number, document
Billing	(event) then GET /v1/accounts/{id}/billing	Charges and payment items from the transaction; invoices by batch	Invoices
Pay	POST /v1/payments	Payment allocated oldest due first; surplus kept as credit	Paid items
Change	Start change, PATCH coverage with If-Match, quote, bind	Copy of latest version; slices split at the effective date; difference premium	Version 2
Claim	POST /v1/claims, exposures, reserves, payments	Policy snapshot as of the loss date kept; payment above limit waits for a second approver	Claim, financials
Missed payment	(event) CancellationRequested	Delinquency steps from configuration; Policy starts cancellation; refund by rule	Cancelled version
Renew	Start renewal	New term of the same policy, rated again; version numbers restart in the term	Term 2

6. Design rules that are always kept

Rule	Why	How it is enforced
Money is whole paise (integers) with a currency	Floating point loses paise; sums must add up exactly	Types, split functions that add up exactly, golden tests
Dates are yyyy-mm-dd; ranges are from-inclusive, to-exclusive	No gaps or double days between slices	to_char in SQL, isRealDate, exclusion constraints
A bound version is never edited	History must be provable	Database triggers refuse changes; changes copy into a branch
Effective-dated rows never overlap	One truth per day	Exclusion constraint plus a clear message from code
Modules never read each other's tables	They stay independent	Architecture test fails the build on a deep import or foreign table
No foreign keys across modules	Modules could be separated later	References checked through the other module's public API
Every change is audited in the same transaction	Accountability	recordAudit inside the transaction; append-only table
Business settings are versioned files, never code	Business changes without releases	zod-checked files under config/, new version for each change
No text from a file is ever run as code	Security	Small expression language, no eval
Test data is made up; no secrets in code	Privacy and safety	Seed files, .env ignored by git

7. API, sign-in and screens

API. Every endpoint is one `defineRoute` entry, used both to serve the request and to generate the OpenAPI 3.1 description at `/openapi.json`, so the documentation cannot drift from the behaviour. Rules: every POST needs an `Idempotency-Key` (a repeat gets the stored answer and the work happens once); changes to shared things need `If-Match` with the ETag; lists are paged; every error has one shape; business rules live in modules, never in routes.

Sign-in. People sign in with an identity provider, which issues a signed token (JWT). The API checks the signature against the provider's published keys, the issuer, the audience and the expiry, accepts only asymmetric algorithms, and gives the same plain 401 for every failure. The token proves *who*; our database says *what they may do* (roles, permissions, producer codes) and whether the user is active. A development mode trusting an `X-User` header exists for tests and local runs; with a real database the default is token mode. A **permissions matrix test** builds, from the route list and role definitions, who may call what, for every route and every role.

Screens. A React quote wizard and a policy file talk only to the REST API. They contain no price, rule or limit of their own, show the server's reasons for a refusal, and disable a button while its action runs (keeping its idempotency key until success).

8. Configuration, database and tests

Configuration family	Folder	Holds
Typelists	config/typelists	Code lists (reasons, causes) with English, Hindi and Telugu names; codes are retired, never removed
Products	config/products	Private car product: coverages, terms, availability
Rate books	config/ratebooks	Rates in basis points, amounts in paise, one per product version
Underwriting	config/underwriting	Rules and authority profiles
Cancellation	config/cancellation	Reasons, notice, refund methods, short-rate table, reinstatement window
Renewal	config/renewal	Lead days, non-renewal notice
Billing	config/billing	Payment plans, grace days, delinquency steps

Configuration family	Folder	Holds
Claims	config/claims	Cover needed per cause of loss, authority limits

Database. The code uses one small `Database` interface (query, exec, transaction, close) with two implementations. 23 plain SQL migrations create one schema per module; an applied migration is never edited (checksums detect it). The same migrations and the same suite run on PGlite and on a real PostgreSQL server; running on the real server found three differences PGlite hid (UTF8 encoding, per-connection settings, parallel database creation).

Tests. Tests are written first and sit beside the code. Layers: unit tests; golden tests for rating; state-move tests for every allowed and forbidden job transition; a scenario test that follows one customer through every module and checks cross-module invariants (premium equals charges, billed minus paid, refunds never exceed cash); an architecture test for module boundaries; API tests; a permissions matrix; screen tests with jsdom. `npm run check` runs the type check, the linter and all tests and must pass before every commit.

9. Running and deploying

Goal	Command
Everything checked	npm run check
Tests on real PostgreSQL (started for the run)	TEST_DATABASE=postgres npm test
API with local data (demo users)	npm run api (port 3000)
Screens	npm run web (port 5173, passes /v1 to the API)
PostgreSQL + API in Docker	cp .env.example .env ; docker compose up --build

Server settings (environment): `DATABASE_URL`, `AUTH_MODE` (oidc or header), `OIDC_ISSUER`, `OIDC_AUDIENCE`, `OIDC_JWKS_URL`, `SEED_DEMO`, `PORT`, `HOST`. A Dockerfile (two stages, health check), a compose file and a CI workflow (check, PostgreSQL tests, image build) are written but have **not been executed**: Docker is not installed here and the project has no Git remote yet.

10. What is done, what is missing

Area	Status
Learning roadmap A1-I2 (29 steps)	Done and committed; about 55% of the full implementation document by section scorecard (REVIEW.md)
Tier 1.1 real PostgreSQL, Docker, CI	Code and tests done on both databases; Docker, CI and deployment unrun
Tier 1.2 real sign-in, permissions matrix	Done with a stand-in identity provider; no real provider, no login screen, no API route for approving underwriting issues
Tier 1.3 (next)	Event publisher with retries and dead-letter; locks for idempotency keys; optimistic locking at bind
Tier 1.4 (next)	Claims history feeding renewal and no-claim bonus; cancellation netting as a small ledger
Tiers 2-4 (REVIEW.md)	More products, document and notification handling, reporting, operations, scale and security hardening
Not in scope of the learning version	Multiple lines of business, payment gateways, real hosting in India, secrets vault, backups, TLS

Companion files in the project: README.md (commands), PROGRESS.md (what is built), LEARNING_NOTES.md (concepts in simple words), REVIEW.md (comparison with the document and the extension plan), CLAUDE.md (the rules every change follows).